Metro Ridership Forecasting using Inter-Station-Aware Transformer Networks

Big Data Course Presentation

Maksym DOLHOV Mehdi AGHAEI Hồ Báo Khánh Nguyễn Nima DAVARI

Big Data Course

Presentation Roadmap

- **Part 1: Problem Context & Research Gap**
  The challenges of metro forecasting and the paper's core idea.
- **Part 2: The ISA-TF Solution**
  3-View Graph Formulation and Transformer Architecture.
- **Part 3: Experimental Evaluation**
  Datasets, Baselines, and State-of-the-Art (SOTA) comparisons.
- **Part 4: Deep Dive & Conclusion**
  Parameter Efficiency, Ablation Study, and Key Takeaways.

[1/12] Problem Context (Presenter 1: Maksym DOLHOV)

- Metro systems need station-level inflow/outflow forecasts for operations.
- Accurate forecasts improve:
  - train scheduling
  - staffing
  - maintenance planning
- Poor forecasts $\rightarrow$ congestion and delays
- The task is hard because demand is both **temporal** and **network-dependent**.

[2/12] Research Gap and Paper Idea (Presenter 1: Maksym DOLHOV)

- Prior methods include statistical, machine learning, and hybrid approaches.
- Many RNN/GCN-based methods are strong for short horizon but can degrade for longer horizon.
- This paper proposes **ISA-TF**: a unified transformer framework for metro ridership forecasting.
- Core idea: fuse ridership history with inter-station topology information.

[3/12] Claimed Contributions (Presenter 1: Maksym DOLHOV)

- New **Inter-Station-Aware Transformer Networks** architecture.
- Uses three metro graph views: physical, semantic similarity, and correlation.
- Evaluated on SHMetro and HZMetro benchmarks.
- Reports stronger long-horizon performance and a much lower parameter count.

[4/12] Problem Formulation (presenter 2: Soroosh)

- Input: past ridership sequence and metro graph:

$$\{D^N_{t-n+1}, \ldots, D^N_t\},\ G \rightarrow \{\hat{D}^N_{t+1}, \ldots, \hat{D}^N_{t+m}\}$$

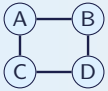
- Each station signal contains two values: inflow and outflow.
- In experiments: $n = 4$ past intervals (60 min total), $m = 4$ future intervals.
- Inference is autoregressive, so horizons longer than 60 min are possible.

[5/12] Metro Graph Representation (Presenter 2: Soroosh)

Three Complementary Graph Views

Physical Graph

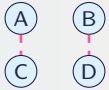
Adjacent tracks



Local spillover between neighbors

Semantic Graph

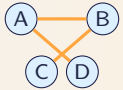
Similar behaviors



Distant stations can share demand patterns

Correlation Graph

Actual travel routes

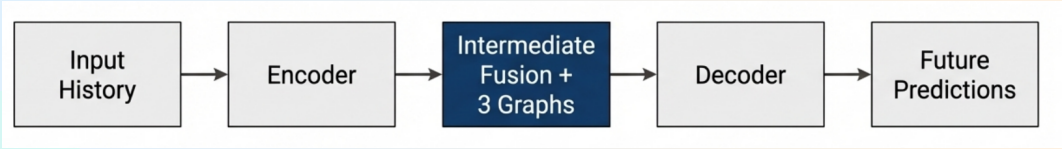


Origin-Destination (OD) flows reveal where passengers actually move

ISA-TF combines M_p, M_s, and M_c because physical adjacency alone does not fully explain crowd dynamics.

[6/12] ISA-TF Architecture (Presenter 2: Soroosh)

- Transformer encoder–decoder backbone
- Encoder processes historical ridership
- Graph information fused using:
  - M_p physical topology
  - M_s semantic similarity
  - M_c correlation matrix
- Decoder predicts future ridership autoregressively



ISA-TF Architecture Pipeline

[7/12] Datasets (Presenter 3: Khanh)

Property	SHMetro	HZMetro
Collection period	Jul–Sep 2016	Jan 1–25, 2019
Stations	288	80
Physical edges	958	248
Avg daily passengers	8.82M	2.35M
Sampling interval	15 minutes	15 minutes

Raw transaction records include: passenger ID, entry/exit station, and timestamps. SHMetro contains 811.8M transactions.

[8/12] Experimental Setup (Presenter 3: Khanh)

- Preprocessing creates the three graph types per dataset.
- Training details: Adam optimizer, 250 epochs, L_2 loss.
- Model hyperparameters: embedding size 512, 8 attention heads.
- Evaluation metrics: RMSE(Root Mean Square Error) and MAE(Mean Absolute Erro).
- Baselines: HM(Historical Mean),
  GB(Gradient Boosting),
  MLP(Multiple Layer Perceptron),
  RNN-GRU(recurrent neural network-Gated Recurrent Unit, it consists of two GRU layers with its hidden units set to 256.),
  STG2Seq(Spatial-temporal Graph to Sequence),
  DCRNN(Diffusion Convolutional Recurrent Neural Network),
  PVCGN(Physical-Virtual Collaboration Graph Network).

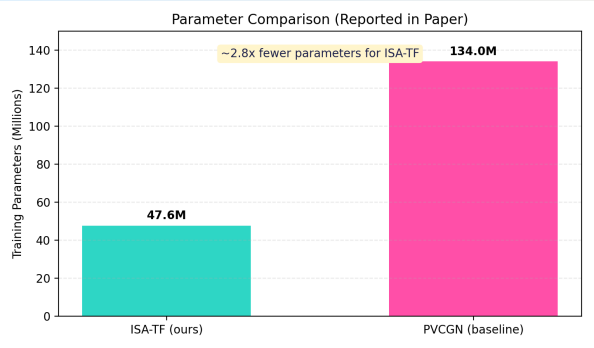
[9/12] Main Results (Presenter 3: Khanh)

Dataset	Horizon	ISA-TF (RMSE/MAE)	PVCGN (RMSE/MAE)
SHMetro	60 min	**48.43/23.87**	55.27/26.29
HZMetro	60 min	**41.68/24.91**	42.61/24.93
SHMetro	15 min	45.85/**23.05**	**44.97**/23.29

- Strong long-horizon robustness is the key result.
- Short horizon remains competitive with the prior SOTA(State-of-the-Art).

Key Result: ISA-TF performs best on long-horizon prediction (45–60 min).

[10/12] Efficiency Comparison (Presenter 4: Nima)



Explanation: Based on the paper's reported values, ISA-TF uses ∼47.6M parameters while the prior state-of-the-art (PVCGN) requires ∼134M.

Why this matters: ISA-TF is about **2.8x lighter**. This means faster training times, lower memory consumption, and easier deployment on city transit servers, all while achieving better long-horizon accuracy.

Source: values reported by the paper (Figure 2 discussion and results section).

[11/12] Do We Need All 3 Graphs? (Presenter 4: Nima)

Question: Is the complex 3-graph fusion actually necessary?

The paper tested ISA-TF by systematically removing components. The results proved that relying on just one type of graph is insufficient for urban forecasting:

- **Without Physical Graph (M_p):** The model fails to predict immediate spillover congestion to next-door stations.
- **Without Semantic Graph (M_s):** The model struggles to link geographically distant stations that share similar commuting demographics (e.g., two business districts).
- **Without Correlation Graph (M_c):** The model misses the actual human origin-destination travel flows, losing accuracy during major events.

Conclusion: The synergy of all three graphs is the secret to ISA-TF's long-term accuracy.

[12/12] Conclusion & Key Takeaways (Presenter 4: Nima)

- **The Problem:** Long-horizon metro forecasting is difficult because passenger flow relies on complex spatial and temporal rules.
- **The Solution (ISA-TF):** Fuses the long-memory power of **Transformers** with a **3-Graph topology network** (Physical, Semantic, Correlation).
- **The Result:** Matches or beats prior SOTA models at 15-minute horizons, and significantly outperforms them at the 45–60 minute horizons.
- **The Bonus:** Achieves this with nearly **3x fewer parameters** than the previous top model.

Reference: K. Saleh, A.-S. Mihaita, and Y. Ou, 2023 IEEE ITSC.

Thank You

Questions?

1.1 What is Time-Series Data?

Time-Series is data collected over time, in chronological order.
The paper frames ridership prediction as a time-series forecasting problem. Imagine counting people at a door every 15 minutes.

- t represents the *current* time interval (e.g., 10:00 AM).
- $t-1$ is the *previous* interval (9:45 AM).
- $t+1$ is the *next* interval (10:15 AM).

When the paper says it uses $n = 4$ historical steps to predict $m = 4$ future steps, it means: *"I will look at the last 60 minutes (four 15-min chunks) to guess the next 60 minutes."*

1.2 The Problem: Time-Series Forecasting

What are we trying to do? Predict how many passengers will enter and exit a metro station in the future based on past data.

The Notation (The Math):

- Past data sequence (length n): $(D^N_{t-n+1}, D^N_{t-n+2}, ..., D^N_t)$.
- Future data sequence (length m): $(\hat{D}^N_{t+1}, \hat{D}^N_{t+2}, ..., \hat{D}^N_{t+m})$.
- N is the total number of stations.
- $D \in \mathbb{R}^2$ means the data has two values:
  - **Inflow Count:** People entering the station to ride.
  - **Outflow Count:** People leaving the station into the city.

2.1 What is a Neural Network?

Goal: Learn a function that maps inputs to outputs.
Example in this paper:

$$\text{Past ridership data} \rightarrow \text{Future ridership prediction}$$

A neural network is a stack of mathematical functions $y = f(x)$, where:

- $x =$ input data
- $f =$ learned function
- $y =$ predicted output

Instead of manually designing the function, the network **learns it from data**.

2.2 Weights and Parameters

Neural networks learn by adjusting internal numbers called **parameters** or **weights**.

Example of a simple neuron:

$$y = w_1 x_1 + w_2 x_2 + b$$

Where:

- x_1, x_2 = input features
- w_1, w_2 = weights (parameters to learn)
- b = bias

During training, the model changes these weights so predictions become more accurate.

2.3 Forward Pass: Making a Prediction

Training a neural network starts with a **forward pass**.
Steps:

1. Input data enters the network.
2. Each layer applies a mathematical transformation.
3. The network produces a prediction.

Example:

$$\text{Past metro data} \rightarrow \text{Neural network} \rightarrow \hat{y} \text{ (predicted ridership)}$$

At this stage, the prediction is usually wrong because the weights are not yet trained.

2.4 Loss Function: Measuring Error

After making a prediction, we measure how wrong it is using a **loss function**.

Example used in the paper: L2 loss

$$L = (\hat{y} - y)^2$$

Where:

- y = true historical value
- $\hat{y}$ = AI's predicted value

Goal of training: **Minimize the loss.** The smaller the loss, the better the prediction.

3.1 What is Backpropagation?

The Concept: If the "Forward Pass" is guessing, **Backpropagation** is learning from the mistake. It sends the error backwards to update the weights.

```
1. FORWARD (Guessing):
   Data ---> [Weight A] ---> [Weight B] ---> Guess (100 people)
                                             Real Answer (150)
                                             ERROR = 50

2. BACKWARD (Learning via Backpropagation):
   Data <--- [Update A] <--- [Update B] <--- ERROR = 50
             (Blame A)       (Blame B)
```

- It uses the **Chain Rule** from calculus to calculate "Gradients".
- It asks: *"How much did Weight B contribute to the error? How about A?"*
- The Optimizer uses this "blame" to tweak the weights.

3.2 Gradients and Backpropagation (The Math)

To improve predictions, the network adjusts its weights using **gradients**.
A gradient tells us:

"If we slightly change this weight, how will the loss change?"

Mathematically:

$$\frac{\partial L}{\partial w}$$

The exact Backpropagation sequence: 1. Compute prediction (Forward Pass) $\rightarrow$ 2. Measure loss $\rightarrow$ 3. Compute gradients (Backward Pass) $\rightarrow$ 4. Update weights.
This repeats thousands of times.

3.3 How AI Learns: The "Mountain" Analogy

Imagine the AI is standing at the top of a mountain, blindfolded.

- **Top of the mountain:** 100% wrong (high error/loss).
- **Bottom of the valley:** 100% right (zero error).

The AI wants to walk down to the bottom.

- The **Loss Function** tells the AI how high up the mountain it currently is.
- The **Optimizer** is the strategy the AI uses to feel the ground with its foot and decide which direction to step next to go downhill.

3.4 What is the Adam Optimizer?

An **Optimizer** is the mathematical engine that updates the weights based on the gradients.
Adam (Adaptive Moment Estimation) is a popular, smart optimizer.

- Instead of taking the same size step every time, Adam adapts.
- If the mountain is steep, Adam takes big, fast steps.
- If the AI feels it is getting close to the bottom (the right answer), Adam automatically takes tiny, careful steps so it doesn't overshoot the target.

3.5 Epochs and The Training Loop

What is an Epoch? Think of the historical metro dataset as a giant stack of flashcards. An **Epoch** is exactly ONE full pass through the entire stack. This model trained for **250 epochs** (it reviewed the history 250 times).

The Full Training Loop:

1. **Guess:** AI looks at 60 mins of past data, guesses next 60 mins.
2. **Check Error:** Compares guess to the real answer (L2-loss).
3. **Learn:** Backpropagation finds gradients; Adam tweaks weights.
4. **Repeat:** Do this for all data (1 Epoch). Repeat 250 times!

4.1 How Gradients Flow & Why They Vanish

During training, the error signal travels backward. Each layer multiplies the gradient by a fraction. If many layers are stacked, the gradient shrinks rapidly.

```
Layer 4 gradient = 1.0 × 0.5 = 0.5
Layer 3 gradient = 0.5 × 0.5 = 0.25
Layer 2 gradient = 0.25 × 0.5 = 0.125
Layer 1 gradient = 0.125 × 0.5 = 0.0625  <-- Almost Zero!
```

The Vanishing Gradient Problem: When gradients become extremely small near the input layers:

- Weight updates become tiny.
- Early layers stop learning entirely.
- Training becomes very slow or stalls.

4.2 Activation Functions in Neural Networks

Motivation: Neural networks naturally apply simple linear transformations (straight lines). If only linear operations were used, the network could never learn complex patterns (like sudden crowd spikes at a metro).

Solution: Activation Functions

- Activation functions introduce **nonlinearity**.
- They act as "gates" deciding if a neuron should fire or not.
- Without them, deep networks are useless; with them, networks can approximate almost any complex real-world data curve.

4.3 Sigmoid vs. ReLU

Older models used the **Sigmoid** function (an S-curve between 0 and 1). At the flat edges of the curve, the slope is nearly 0. This caused massive Vanishing Gradients.
The Fix: ReLU (Rectified Linear Unit)

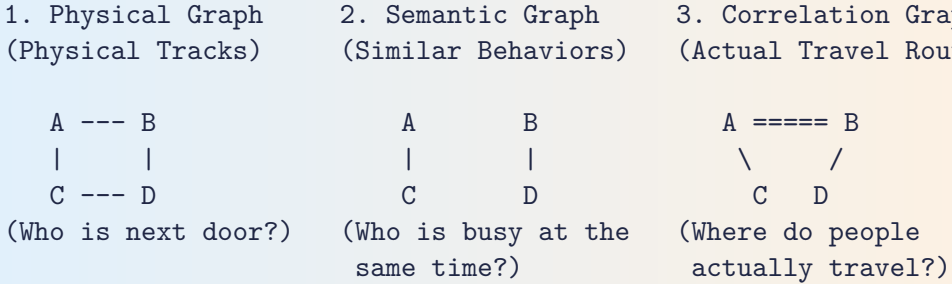
$$ReLU(x) = \max(0, x)$$

- If $x \leq 0$, output is 0. If $x > 0$, output is x.
- Because the slope for positive numbers is a constant straight line, gradients flow perfectly without shrinking.

```
ReLU    |       /     Gradient is constant for positive inputs.
        |      /      This stops gradients from vanishing!
        |_____/
        +--------------
```

5.1 Graph Theory: Mapping the Metro

In Big Data, we represent networks using **Graphs**. This paper uses THREE different graphs to give the AI the full picture:



An **Edge Weight** is a number showing how strongly two stations are connected in these specific views.

5.2 Physical (M_p) & Semantic (M_s) Graphs

Physical Edge Weight (M_p): Are these stations directly connected by a train track?

- Base score is 1 if connected, 0 if not. Formula standardizes it to fractions.
- *Why it matters:* Captures direct crowd spillover to the next station.

Semantic Similarity (M_s) & DTW: Do these stations have the same "vibe"? (e.g., two business districts far apart that both peak at 5:00 PM).

- Uses **Dynamic Time Warping (DTW)**: An algorithm that matches similar time-series curves even if one is slightly delayed.
- *Why it matters:* Connects stations based on human behavior, not geography.

5.3 Correlation Matrix (M_c)

Correlation Matrix (M_c): Are people actually riding the train directly from Station A to Station B?

- The physical graph shows the tracks.
- The semantic graph shows the schedule/behavior.
- **The correlation graph shows the actual humans.** It counts total travels starting at A and ending at B.

The Math: Takes trips between A and B (R_c) and divides by total trips in network.

$$M_c(a, b) = \frac{R_c(a, b)}{\sum_{k=1}^{N} R_c(a, k)}$$

Why it matters: Directly maps where crowds are flowing in real-time.

6.1 RNNs vs. Transformers

The Old Way: RNNs (Recurrent Neural Networks)

- RNNs read data chronologically, step-by-step.
- *The Flaw:* They struggle to remember old information. They suffer from vanishing gradients over long time horizons.

The New Way: Transformers

- Transformers don't read step-by-step. They look at the entire sequence at once.
- They use **Self-Attention** to score which past time steps are the most important for predicting the future.

6.2 Embeddings: Converting Data into Vectors

Problem: Neural networks cannot process raw concepts, words, or raw time steps easily.
Solution: Embeddings transform each time-step token into a dense vector of real numbers.

- Similar data patterns obtain similar vectors in the mathematical space.
- These embeddings become the starting input used to compute **Query (Q)**, **Key (K)**, and **Value (V)** vectors in the Transformer.

6.3 Unpacking "Self-Attention"

Analogy: Reading a sentence. If you read: "The **bank** of the **river** was muddy." To understand the word "bank", your brain pays *attention* to the word "river" (not a money bank).

Step 1: Q, K, V Vectors From the embedding, the network creates 3 vectors:

- **Query (Q)** – what the token is looking for.
- **Key (K)** – what the token represents.
- **Value (V)** – the actual information the token carries.

Step 2: Attention Scores The model computes similarity by multiplying Query and Key ($Q \cdot K$). Higher scores mean the model should pay more attention to that time step.

6.4 What is Softmax?

The Concept: Softmax is a mathematical function that takes raw, random Attention scores and converts them into **probabilities** that sum to 1.0 (100%).

```
Raw Attention Scores        Softmax Math          Final Probabilities

Time 4:30 PM:  3.2   --->  e^(3.2) / Sum   --->   84%  (Focus here!)
Time 4:45 PM:  1.0   --->  e^(1.0) / Sum   --->   11%  (A little)
Time 4:00 PM: -0.5   --->  e^(-0.5) / Sum  --->    5%  (Ignore)

TOTAL:         3.7                                100%
```

Step 3: Final Output The Softmax probabilities are multiplied by the **Value (V)** vectors. The AI successfully filters out the noise and focuses 84% of its computing power on the 4:30 PM data spike!

6.5 Self-Attention Mechanism (The Math)

Computation Summary:

$$\text{Attention}(Q, K, V) = \mathit{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- Q = Query matrix, K = Key matrix, V = Value matrix.
- The dot product QK^T measures similarity between time steps.
- $\sqrt{d_k}$ scales it down so gradients don't explode.
- Softmax converts to weights (summing to 1).
- Result is multiplied by V to get the final representation.

7.1 The Paper's Invention: ISA-TF

The **Inter-Station-Aware Transformer Networks (ISA-TF)** combines the time-reading power of Transformers with the spatial map of the 3 Graphs.

```
[PAST DATA] --> (ENCODER) --> [FUSION] --> (DECODER) --> [FUTURE DATA]
Last 60 mins   Finds Time   Mixes Time     Guesses the   Next 60 mins
               Patterns     with 3 Graphs    Future
```

- **Encoder:** Uses self-attention to capture time patterns.
- **Fusion:** Fuses encoder output with the 3 metro graphs.
- **Decoder:** Predicts the future auto-regressively.

7.2 Deep Dive: Encoder & Feed-Forward

Encoder:

- Adds "Positional Encoding" so the Transformer knows the order of time (since it doesn't read left-to-right like an RNN).
- Uses Self-Attention to find patterns.
- Passes data through a **Feed-Forward Neural Network (FFN)**:

$$FFN(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

Notice the $\max(0, \dots)$*? That is the ReLU activation function preventing vanishing gradients!*

7.3 Deep Dive: Fusion & Decoder

Intermediate Fusion (Concatenation):

- It simply glues vectors together.
- Example: Time Vector $[0.5, 0.2]$ + Graph Vector $[0.7, 0.1]$ = Fused Vector $[0.5, 0.2, 0.7, 0.1]$.
- Now the AI understands both TIME and SPACE simultaneously.

Decoder:

- Uses **Masked** Self-Attention (so it can't "cheat" and look at future answers during training).
- Uses Cross-Attention to look back at the Fused Vector.
- Predicts the next 15 minutes, feeds that prediction back into itself, predicts the next 15 minutes, etc. (Auto-regressive).

7.4 Deep Dive: What is "Autoregressive"?

The Concept: Instead of predicting the entire 60-minute future all at once in one giant guess, an **autoregressive** model predicts the future step-by-step. It uses its own past predictions as inputs to make the next prediction.

The Step-by-Step Loop (15-min intervals):

```
Step 1: [Real Past 60 mins] --------> Predicts Min 15
Step 2: [Past 45 mins + Min 15] ----> Predicts Min 30
Step 3: [Past 30 mins + Min 30] ----> Predicts Min 45
Step 4: [Past 15 mins + Min 45] ----> Predicts Min 60
```

Analogy: Walking up a staircase in the dark. You have to securely plant your foot on Step 1 before you can reach up to guess exactly where Step 2 is.

7.5 Deep Dive: Autoregressive Python Logic

Why do it this way? It allows the model to predict *any* length of time into the future! However, the risk is **Error Accumulation**: if the Step 1 guess is slightly wrong, Step 2 will be even more wrong because it is based on bad data.

Python Concept:

```
future_predictions = []
current_input = real_past_data  # e.g., 4 steps (60 mins)

# Loop 4 times to predict 1 hour into the future
for step in range(4):
    # Guess the next 15 mins
    next_15 = model.predict(current_input)
    future_predictions.append(next_15)

    # Slide the window forward:
    # Drop oldest real data, append newest AI guess
```

8.1 Evaluation Metrics (RMSE & MAE)

RMSE (Root Mean Square Error): Heavily punishes large errors because differences are squared before averaging. Good for preventing massive congestion mistakes.

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(\hat{D}_i - D_i)^2}$$

MAE (Mean Absolute Error): The simple average of how far off the AI's guesses were.

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|\hat{D}_i - D_i|$$

Note: $\hat{D}_i$ is predicted ridership, D_i is actual ground-truth.

8.2 Decoding the "Baselines" (Competitors)

The paper compares ISA-TF against older models:

- **HM (Historical Mean):** Just averages past data.
- **GB (Gradient Boosting):** A classic ML method combining decision trees.
- **RNN-GRU:** A recurrent network that processes step-by-step (forgets long-term trends).
- **DCRNN / PVCGN:** Advanced models that rely on *Convolutions* and graph networks.

8.3 Why do "Parameters" Matter?

The ISA-TF model uses $\sim$47.6 Million parameters. The previous best model (PVCGN) required $\sim$134 Million.

- Think of parameters as the "gears" inside the AI's engine.
- **Why fewer is better:** A model with fewer parameters trains faster, uses less RAM/electricity, and is much easier for a city to deploy in the real world while getting the exact same or better accuracy!

8.4 Baseline Concept: What is Convolution?

Many baseline models (like DCRNN) use Convolutions instead of Attention.
Definition: Convolution slides a small mathematical filter (kernel) over data to extract local patterns.

- Example Input: $[2, 4, 6, 8]$
- Filter: $[1, 0]$
- It scans: $(2 \cdot 1 + 4 \cdot 0) = 2 \rightarrow (4 \cdot 1 + 6 \cdot 0) = 4 \ldots$

While Convolutions are great for finding local spikes (like an edge in an image), they struggle to connect Station A to Station Z simultaneously, which is why the Transformer's Global Attention usually wins for long-range tasks.

9.1 Equation-Level Mapping for Slide [5/12]

The paper defines three normalized edge-weight matrices (Eq. 1–3):

- **Physical topology graph**

$$M_p(a,b) = \frac{E(a,b)}{\sum_{k=1}^{N} E(a,k)}$$

- **Semantic similarity graph (DTW-based)**

$$M_s(a,b) = \frac{F_s(a,b)}{\sum_{k=1}^{N} F_s(a,k)}$$

- **Correlation graph (Origin–Destination travel interactions)**

$$M_c(a,b) = \frac{R_c(a,b)}{\sum_{k=1}^{N} R_c(a,k)}$$

Interpretation: ISA-TF captures both nearby-station effects and cross-station flow interactions beyond direct physical links.

9.2 Deep Dive: Physical Topology Graph (M_p)

The Concept: Geographical Connections

- This graph represents the actual, physical train tracks.
- If Station A and Station B are next to each other on the same line, they get a base score of 1 ($E(a, b) = 1$).
- If they are not directly connected, they get a 0.

Why does the AI need this?

- **Spillover Effect:** Passenger flow naturally moves down the line. If Station A gets extremely crowded at 8:00 AM, the AI uses this graph to predict that the next physical station down the track will likely see a surge a few minutes later.

9.3 Deep Dive: Semantic Similarity Graph (M_s)

The Concept: Behavioral Connections

- This graph ignores geography and looks at **station function**.
- Example: Two business-district stations on opposite sides of the city might both experience massive outflow at 9:00 AM and massive inflow at 5:00 PM.

How it's calculated: Dynamic Time Warping (DTW)

- DTW is an algorithm that compares the shape of two time-series curves.
- Even if Station A peaks at 5:00 PM and Station B peaks at 5:15 PM, DTW recognizes they share the same "vibe" or pattern and links them mathematically.
- **Why it matters:** It helps the AI learn city-wide demographic habits.

9.4 Deep Dive: Correlation Graph (M_c)

The Concept: Actual Human Movement

- This graph tracks the **Origin-Destination (OD)** pairs.
- It simply counts how many historical passengers tapped their ticket to enter at Station A, and tapped out to exit at Station B.

Why does the AI need this?

- The Physical Graph shows tracks, and the Semantic Graph shows schedules, but the Correlation Graph shows **actual travel paths**.
- If historical data shows that 40% of people who leave a stadium at Station A travel directly to a transfer hub at Station B, this graph gives the AI the exact mathematical percentage it needs to predict the crowd at Station B.

10.1 HM & GB

HM (Historical Mean): The Simplest Baseline

- **How it works:** It looks at the same time and day from the past (e.g., the last four Tuesdays at 5:00 PM) and simply averages the ridership.
- **Why it fails:** It cannot react to sudden, real-time changes (like a train delay or sudden rainstorm causing a crowd).

GB (Gradient Boosting): The Classic ML Approach

- **How it works:** It builds a "Decision Tree" to make a guess. Then, it builds a *second* tree specifically designed to fix the errors of the first tree. It chains hundreds of these trees together.
- **Why it fails here:** It is great for tabular data (like spreadsheets), but it is not naturally built to understand complex spatial maps (Graph theory) or sequential time-series data.

10.2 RNN-GRU

RNN-GRU (Recurrent Neural Network - Gated Recurrent Unit)

- **How it works:** RNNs process data chronologically. The **GRU** part is a special "memory valve" (a gate) that helps the network decide what past information to remember and what to forget.
- **The architecture:** It passes a hidden "state" (memory) from 9:00 AM $\rightarrow$ 9:15 AM $\rightarrow$ 9:30 AM.
- **Why ISA-TF beats it:** Because it processes step-by-step, by the time it reaches the 60th minute, the math has "faded" (vanishing gradient), and it forgets what happened at minute 1. Transformers (ISA-TF) look at all 60 minutes at the exact same time using Attention!

10.3 DCRNN & PVCGN

The Heavyweights: Graph + Convolution Models

- **DCRNN (Diffusion Convolutional Recurrent Neural Network):** Models traffic flowing across the metro graph like heat diffusing through metal. It uses RNNs for time and Graph Convolutions for space.
- **PVCGN (Physical-Virtual Collaboration Graph Network):** The previous State-of-the-Art. It builds both a physical graph (tracks) and a virtual graph (similarity), then uses convolutions to process them.
- **Why ISA-TF beats them:** PVCGN is incredibly heavy (**134 Million parameters**). ISA-TF uses Transformers instead of convolutions, allowing it to capture longer-range time patterns with only a third of the mathematical weight (**47 Million parameters**).

11.1 Python Code: Building the Physical Graph (M_p)

Scenario: We have 3 stations. A connects to B. B connects to A and C. C connects to B.

```
import numpy as np

# Adjacency matrix (1 if track exists, 0 otherwise)
# Stations: A, B, C
E = np.array([
    [0, 1, 0],  # A connects to B
    [1, 0, 1],  # B connects to A and C
    [0, 1, 0]   # C connects to B
])

# Normalize by row sum to get Mp
row_sums = E.sum(axis=1, keepdims=True)
M_p = E / row_sums
```

11.2 Output & Explanation: Physical Graph

Output:

```
[[0.  1.  0. ]
 [0.5 0.  0.5]
 [0.  1.  0. ]]
```

Explanation:

- **Row 1 (Station A):** 100% of its spillover influence goes to B.
- **Row 2 (Station B):** Its spillover is split 50/50 between A and C.
- **Row 3 (Station C):** 100% of its spillover influence goes to B.

This perfectly executes the paper's formula: $M_p(a, b) = \frac{E(a,b)}{\sum E(a,k)}$

11.3 Python Code: Calculating L2 Loss (MSE)

Scenario: AI predicts metro ridership, we compare it to the actual truth.

```
import numpy as np

y_true = np.array([150, 200, 50])  # Actual passengers
y_pred = np.array([140, 210, 60])  # AI Predictions

# Calculate L2 Loss (Mean Squared Error)
errors = y_pred - y_true
squared_errors = errors ** 2
mse = np.mean(squared_errors)

print(f"Errors: {errors}")
print(f"Squared Errors: {squared_errors}")
print(f"MSE (L2 Loss): {mse}")
```

11.4 Output & Explanation: L2 Loss

Output:

```
Errors: [-10  10  10]
Squared Errors: [100 100 100]
MSE (L2 Loss): 100.0
```

Explanation:

- The AI was off by exactly 10 passengers for each station.
- We square the errors ($-10 \times -10 = 100$) so negative and positive errors don't cancel each other out to zero.
- The average of $[100, 100, 100]$ is 100.
- The **RMSE**(root mean squared error) would simply be $\sqrt{100} = 10$ passengers off on average.

11.5 Python Code: Self-Attention ($Q \times K^T$)

Scenario: Calculate unnormalized attention scores for 2 time steps.

```
import numpy as np

# Time 1 (T1) and Time 2 (T2) Embeddings
# Q and K are 1x2 vectors for each time step
Q = np.array([[1.0, 0.5],   # T1 Query
              [0.0, 1.0]])  # T2 Query

K = np.array([[1.0, 0.0],   # T1 Key
              [0.0, 1.0]])  # T2 Key

# Dot Product: Q * K^T (Matrix Multiplication)
scores = np.dot(Q, K.T)
print(scores)
```

12.1 Math by Hand: Softmax Calculation

Scenario: We have raw attention scores for Station A: $x_1 = 2.0$, $x_2 = 1.0$, $x_3 = 0.0$. Let's convert them to percentages.

Step 1: Calculate Exponentials (e^x) (Assume $e \approx 2.718$)

- $e^{2.0} \approx 7.39$
- $e^{1.0} \approx 2.72$
- $e^{0.0} = 1.00$

Step 2: Sum the Exponentials

$$\Sigma = 7.39 + 2.72 + 1.00 = 11.11$$

Step 3: Divide each by the sum to get probabilities

- $P_1 = 7.39/11.11 \approx 0.665$ (**66.5%**)
- $P_2 = 2.72/11.11 \approx 0.245$ (**24.5%**)
- $P_3 = 1.00/11.11 \approx 0.090$ (**9.0%**)

12.2 Math by Hand: $Q \times K^T$ (Dot Product)

Goal: Understand how the dot product finds similarity between time sequences. Suppose Query for Time 1 is $Q_1 = [2, 1]$ and Key for Time 2 is $K_2 = [1, 1]$.

The Dot Product Formula:

$$Q_1 \cdot K_2 = (Q_{1,1} \times K_{2,1}) + (Q_{1,2} \times K_{2,2})$$

Calculation:

$$(2 \times 1) + (1 \times 1) = 2 + 1 = 3$$

Interpretation: The similarity score is 3. If K_2 was completely opposite, say $[-2, -1]$, the score would be -5. The higher the positive number, the more the AI pays *attention* to that specific time step!

13.1 Active Learning: Test Your L2 Loss Knowledge

Question 1: Calculate the MSE (L2 Loss) by hand.
You are testing the ISA-TF model on two stations for the 8:00 AM interval.

- **Ground Truth (y):** Station A = 500, Station B = 300
- **AI Prediction ($\hat{y}$):** Station A = 480, Station B = 310

Use the formula:

$$MSE = \frac{1}{n}\sum(\hat{y}_i - y_i)^2$$

Task: Find the errors, square them, and find the mean. (Answer on next slide).

13.2 Solution: L2 Loss Calculation

Step 1: Calculate raw errors

- Station A error: $480 - 500 = -20$
- Station B error: $310 - 300 = 10$

Step 2: Square the errors

- Station A squared: $(-20)^2 = 400$
- Station B squared: $(10)^2 = 100$

Step 3: Average them (Mean)

$$MSE = \frac{400 + 100}{2} = \frac{500}{2} = \mathbf{250}$$

Bonus: The RMSE (Root Mean Square Error) is $\sqrt{250} \approx 15.8$ passengers.

Question 2: Calculate the Physical Graph Matrix row by hand.
Station X is connected to Station Y, Station Z, and Station W by physical tracks. It is NOT connected to Station V.

Base edge weights (E):

- $E(X, Y) = 1$
- $E(X, Z) = 1$
- $E(X, W) = 1$
- $E(X, V) = 0$

Task: Calculate the normalized weights $M_p(X, Y)$ and $M_p(X, V)$ using:

$$M_p(a, b) = \frac{E(a, b)}{\sum_{k=1}^{N} E(a, k)}$$

13.4 Solution: Graph Normalization

Step 1: Find the denominator (Sum of all connections for Station X)

$$\sum E = E(X, Y) + E(X, Z) + E(X, W) + E(X, V)$$

$$\sum E = 1 + 1 + 1 + 0 = 3$$

Step 2: Calculate specific probabilities

- For Y: $M_p(X, Y) = \frac{1}{3} \approx 0.33$ (**33.3% spillover**)
- For V: $M_p(X, V) = \frac{0}{3} = \mathbf{0}$ (**0% spillover**)

Takeaway: The model now mathematically knows that 33% of Station X's spatial influence goes directly to Station Y, but none goes to V.

13.5 Active Learning: Test Your Softmax Intuition

Question 3: Softmax logic without a calculator
You have raw attention scores for three past time intervals (T1, T2, T3). The raw scores outputted by the dot product are: $T1 = 5.0,\ T2 = 5.0,\ T3 = -10.0$.

Task (Use logic, not math!):

1. Which time interval will get almost **zero** attention probability?
2. What will the approximate probability percentage be for T1 and T2?

13.6 Solution: Softmax Intuition

1. **Which interval gets almost zero attention?**
   - **T3**. Because -10.0 is exponentially smaller than 5.0. In Softmax math, e^{-10} is practically zero. The AI will completely ignore T3.

2. **What is the approximate probability for T1 and T2?**
   - **50% each**. Since $T1 = 5.0$ and $T2 = 5.0$, their exponential values are exactly equal.
   - Because T3 is mathematically ignored (0%), the remaining 100% attention budget is split perfectly in half between T1 and T2.

Takeaway: Softmax acts like a harsh judge. It strongly amplifies the top values and completely squashes the negative/low values!

13.7 References Used in This Presentation

- K. Saleh, A.-S. Mihaita, and Y. Ou, "Metro Ridership Forecasting using Inter-Station-Aware Transformer Networks," in *2023 IEEE 26th International Conference on Intelligent Transportation Systems (ITSC)*, pp. 1215–1220, 2023.
  Used for: the main paper summary, metro graph formulation, ISA-TF pipeline, datasets, quantitative results, and parameter comparison.
- L. Liu, J. Chen, H. Wu, J. Zhen, G. Li, and L. Lin, "Physical-virtual collaboration modeling for intra-and inter-station metro ridership prediction," *IEEE Transactions on Intelligent Transportation Systems*, 23(4):3377–3391, 2020.
  Used for: the PVCGN baseline and the physical/semantic/correlation graph viewpoint reused in the paper discussion.
- A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in Neural Information Processing Systems*, 30, 2017.
  Used for: the appendix explanation of transformers, self-attention, Q/K/V, and softmax.
- Y. Li, R. Yu, C. Shahabi, and Y. Liu, "Diffusion convolutional recurrent neural network: Data-driven traffic forecasting," *ICLR 2018*, 2018.
  Used for: the DCRNN comparison slide and the discussion of graph-convolution baselines.
- L. Bai, L. Yao, S. S. Kanhere, X. Wang, and Q. Z. Sheng, "Stg2seq: spatial-temporal graph to sequence model for multi-step passenger demand forecasting," in *IJCAI 2019*, pp. 1981–1987, 2019.
  Used for: the baseline table and appendix comparison of earlier sequence/graph forecasting models.